Comparação de estruturas de dados

Curso: Ciência da Computação • Professor: Daniel Gomes Soares
Realização: individual ou em dupla

1 Objetivo

Desenvolva um programa que execute as mesmas operações em diferentes estruturas de dados, meça o tempo gasto e apresente uma comparação dos resultados.

2 Quais estruturas utilizar

Estrutura	Implementação a utilizar
Array linear	Array de inteiros não ordenado, com busca sequencial.
Array ordenado	Array mantido em ordem crescente, com busca binária.
Lista duplamente encadeada	Implementação desenvolvida na disciplina.
Tabela hash	Implementação desenvolvida na disciplina.
Árvore binária de busca	Implementação sem balanceamento automático.
Árvore AVL	Implementação desenvolvida na disciplina.
Árvore rubro-negra	Implementação desenvolvida na disciplina.

Utilize suas próprias implementações, aproveitando e ajustando os códigos feitos nas aulas. Não utilize ArrayList, LinkedList, HashSet, HashMap, TreeSet ou TreeMap para representar as estruturas comparadas. Os métodos de inserção, busca, remoção e consulta por intervalo devem ser implementados nos seus códigos.

Bibliotecas Java podem ser usadas nas tarefas auxiliares: gerar e organizar os dados de entrada, medir o tempo, gravar arquivos e calcular estatísticas. Para fazer gráficos, podem ser usados Excel, LibreOffice, Python ou outra ferramenta.

3 Como preparar os dados

- Use três quantidades iniciais de elementos: 5.000, 10.000 e 100.000. Chamaremos essa quantidade de n.
- Para cada n, prepare os números pares de 2 até 2n. Exemplo: para n = 5.000, os valores serão 2, 4, 6, ..., 10.000.
- Prepare três ordens de entrada com os mesmos valores: crescente, decrescente e aleatória. A ordem indica como os valores serão inseridos na estrutura.
- Use a semente 42 na geração aleatória. Para cada tamanho e ordem testados, reaproveite os mesmos dados e sequências de operações em todas as estruturas e repetições.

Todas as estruturas devem armazenar valores sem repetição. Uma tentativa de inserir um valor já existente deve ser ignorada. A verificação de duplicatas faz parte da inserção, inclusive nos arrays e na lista.

4 Quais testes executar

Execute os seis testes abaixo em cada estrutura, para os três tamanhos e as três ordens de entrada. Um lote é o conjunto de operações de um teste. Cronometre o lote completo.

Teste	O que fazer em cada execução
Construção	Comece com a estrutura vazia e insira os n valores, um a um, na ordem definida. Meça desde a criação da estrutura até o fim das inserções.
Busca presente	Com os n valores já armazenados, faça 2.000 buscas por valores sorteados desse conjunto. Um mesmo valor pode ser consultado várias vezes.
Busca ausente	Com os n valores já armazenados, faça 2.000 buscas por números ímpares sorteados entre 1 e 2n − 1. Eles não pertencem ao conjunto inicial.
Inserção	Com os n valores já armazenados, insira mais 10% de n elementos. Sorteie ímpares distintos entre 1 e 2n − 1 e insira-os na ordem sorteada.
Remoção	Com os n valores já armazenados, remova 10% de n elementos distintos, sorteados do conjunto inicial. Inclua no tempo a busca pelo valor e sua remoção.
Consulta por intervalo	Com os n valores já armazenados, faça 200 consultas. Em cada uma, conte quantos valores estão entre mínimo e máximo, incluindo os limites. Sorteie mínimo entre 1 e 2n − 100 e use máximo = mínimo + 100.

Cada teste é independente. Antes de cada execução, reconstrua a estrutura com os n valores iniciais, exceto no teste de construção, que começa vazio. As inserções de um teste não devem alterar os dados utilizados nos outros testes.

Exemplo: para n = 5.000, o lote de inserção adiciona 500 valores e termina com 5.500 elementos. O lote de remoção parte novamente de 5.000 elementos, remove 500 e termina com 4.500.

5 Como medir e repetir

Para cada combinação de estrutura, tamanho, ordem e teste, execute uma vez sem registrar o tempo, como aquecimento. Depois, execute mais cinco vezes e guarde os cinco tempos. Reconstrua o estado inicial antes de cada execução, inclusive após o aquecimento.

Gere os sorteios antes da medição. Impressões na tela, gravação de arquivos e conferência dos resultados também devem ficar fora do trecho cronometrado.

6 Cuidados para comparar corretamente

- Confira os resultados antes de comparar os tempos: buscas presentes devem encontrar o valor; buscas ausentes devem retornar falso; inserções, remoções e intervalos devem produzir o mesmo resultado em todas as estruturas.
- Guarde e use o resultado das operações, por exemplo somando as buscas bem-sucedidas ou as quantidades retornadas nos intervalos. Mostre essa conferência depois de parar o cronômetro.
- Execute todos os testes no mesmo computador, sem o depurador, e evite outros programas pesados durante as medições. Informe no relatório o processador, a memória e a versão do Java.
- Reserve nos arrays espaço para n mais 10% de n elementos. Mantenha a política de capacidade da hash igual nas repetições e informe como ela trata colisões e amplia sua capacidade. O custo de uma ampliação durante uma operação entra no tempo medido.
- Construa o array ordenado mantendo a ordenação a cada inserção. O custo de inserir inclui localizar a posição e deslocar os elementos necessários.

7 Como organizar os resultados

Produza dois arquivos CSV:

- resultados_brutos.csv: uma linha por execução, contendo tamanho, ordem, estrutura, teste, número da repetição, quantidade de operações e tempo total em nanossegundos.
- resumo.csv: uma linha por combinação de tamanho, ordem, estrutura e teste, contendo a mediana do tempo total em milissegundos e a mediana do tempo por operação em nanossegundos.

Para obter a mediana, ordene os cinco tempos e escolha o terceiro. Para o tempo por operação, divida o tempo do lote pela quantidade de operações. Exemplo: no teste de busca, divida por 2.000. Grave números sem separador de milhares; separe as colunas por ponto e vírgula e use ponto decimal.

8 O que entregar

O relatório técnico deve conter as seguintes seções:

a) Introdução. Apresente o objetivo do trabalho e as estruturas de dados comparadas.

b) Código fonte documentado e arquivos CSVs. Link do GitHub com todas as estruturas, o programa usado nos experimentos e os arquivos CSVs gerados nos experimentos.

c) Análise comparativa. Descreva como os experimentos foram executados e informe a configuração do computador e a versão do Java. Compare todas as estruturas com base nos dados obtidos, apresentando tabelas e gráficos. A análise deve responder:

- Quais estruturas tiveram os menores e os maiores tempos em cada operação?
- Como o aumento da quantidade de elementos afetou os resultados? Relacione com as complexidades esperadas.
- Como a ordem de inserção afetou as árvores? Compare a BST, a AVL e a rubro-negra.
- Quais diferenças apareceram entre o array linear e o ordenado, e entre buscas presentes e ausentes?
- Qual estrutura você escolheria para muitas buscas exatas, para muitas atualizações e para consultas por intervalo? Justifique com os resultados.

Compare as estruturas para o mesmo teste, tamanho e ordem de entrada. Relacione os resultados às características e às complexidades das implementações. Os tempos não precisam coincidir com os exemplos fornecidos.

d) Conclusão. Sintetize os principais resultados e o que foi aprendido sobre as vantagens e limitações das estruturas comparadas.

Comentários adicionais

a) Valores repetidos não são permitidos. Garanta que cada estrutura tenha exatamente n valores distintos ao iniciar os testes, exceto no teste de construção, que começa vazio, e respeite as quantidades previstas para inserção e remoção.

b) A recursividade pode provocar estouro de pilha em conjuntos grandes, especialmente na árvore binária de busca sem balanceamento. Se isso ocorrer, adapte o método para uma versão não recursiva, repita os testes afetados e documente o ajuste no relatório